Section A:

Section A1:

In [1]:
```
#1
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import copy
import sklearn

#2
df = pd.read_csv("SeoulBikeData_mod.csv")

#3
df.info()
df.describe()
```

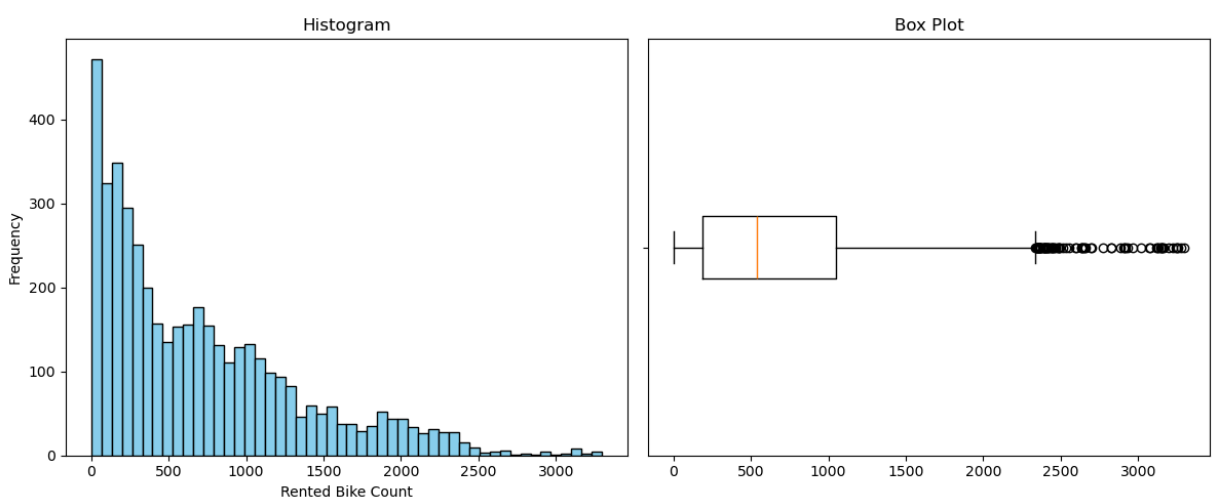
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4416 entries, 0 to 4415
Data columns (total 11 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Rented Bike Count          4416 non-null   int64
 1   Hour                       4416 non-null   int64
 2   Temperature(°C)            4416 non-null   float64
 3   Humidity(%)                4416 non-null   int64
 4   Wind speed (m/s)           4416 non-null   float64
 5   Visibility (10m)           4416 non-null   int64
 6   Dew point temperature(°C)  4416 non-null   float64
 7   Solar Radiation (MJ/m2)    4416 non-null   float64
 8   Rainfall(mm)               4416 non-null   float64
 9   Snowfall (cm)              4416 non-null   float64
 10  Month                      4416 non-null   int64
dtypes: float64(6), int64(5)
memory usage: 379.6 KB
```

Out[1]:

	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibility (10m)
count	4416.000000	4416.00000	4416.000000	4416.000000	4416.000000	4416.000000
mean	704.591259	11.50000	13.203986	59.257246	1.653193	1398.641304
std	637.407244	6.92297	11.529724	20.162981	0.976361	621.560536
min	0.000000	0.00000	-17.800000	0.000000	0.000000	27.000000
25%	189.000000	5.75000	4.500000	44.000000	0.900000	853.000000
50%	542.000000	11.50000	14.300000	58.000000	1.500000	1621.000000
75%	1047.000000	17.25000	22.200000	75.000000	2.200000	1999.000000
max	3298.000000	23.00000	38.000000	98.000000	6.700000	2000.000000

In [2]:

```
#4
RentedBikeCount = df["Rented Bike Count"]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

ax1.hist(RentedBikeCount, bins=50, color="skyblue", edgecolor="black")
ax1.set_xlabel("Rented Bike Count")
ax1.set_ylabel("Frequency")
ax1.set_title("Histogram")

ax2.boxplot(RentedBikeCount, vert=False, tick_labels=[""])
ax2.set_title("Box Plot")

plt.tight_layout()
plt.show()
```



In [3]:

```
#5
X = df.drop(columns = ["Rented Bike Count"])
y = df["Rented Bike Count"]

#6
```

```
# in the research paper, 75% of the data is used for training and 25% is used for t
X_train, X_test, y_train, y_test = sklearn.model_selection.train_test_split(X, y, t

#7
model = sklearn.linear_model.LinearRegression()
model.fit(X_train,y_train)

#8
y_pred = model.predict(X_test)
resid = y_test - y_pred

#9
R2 = sklearn.metrics.r2_score(y_test, y_pred)
MAE = sklearn.metrics.mean_absolute_error(y_test, y_pred)
MSE = sklearn.metrics.mean_squared_error(y_test, y_pred)
RMSE = np.sqrt(MSE)

CV = (RMSE/np.mean(y_test))*100

print(f"R2 = {R2}")
print(f"MAE = {MAE}")
print(f"RMSE = {RMSE}")
print(f"CV = {CV}")
```

```
R2 = 0.4864602577706074
MAE = 330.20376773244544
RMSE = 447.88670296177287
CV = 65.38215352013525
```

Seoul Bike Paper results:

R2 = 0.55

MAE = 322.32

RMSE = 427.71

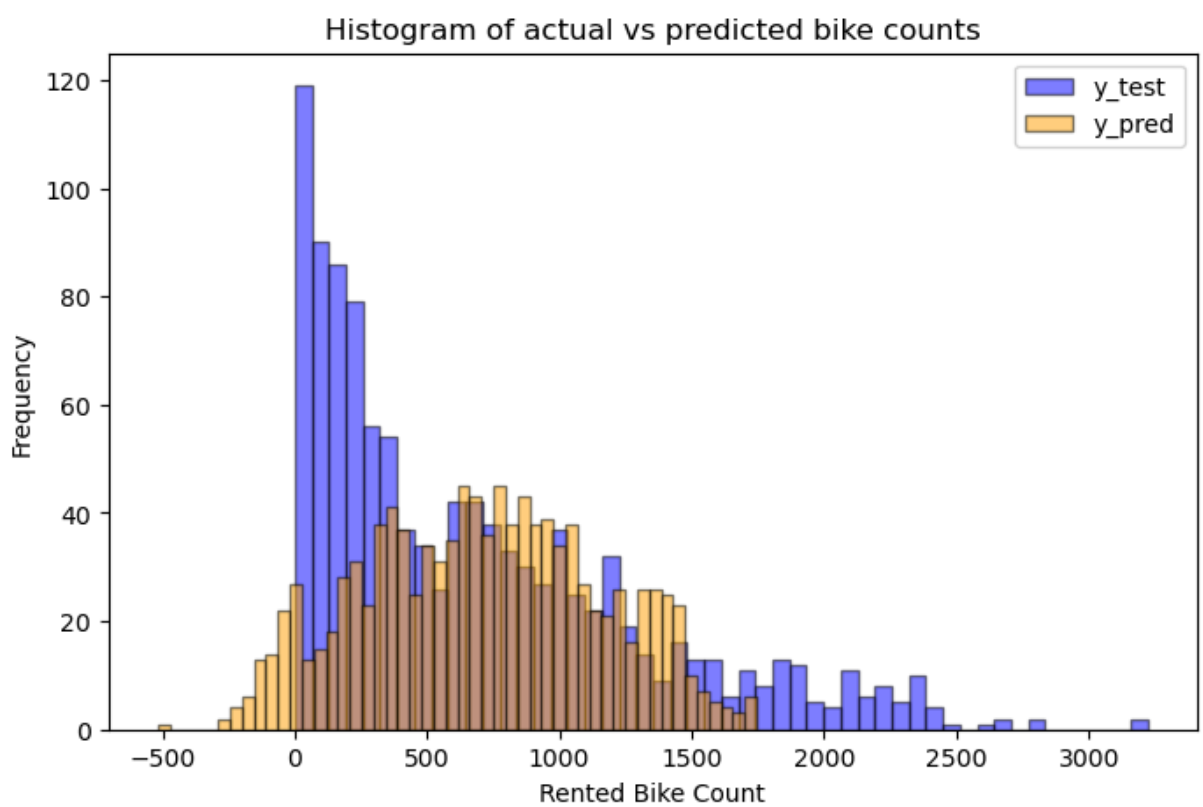
CV = 61.03

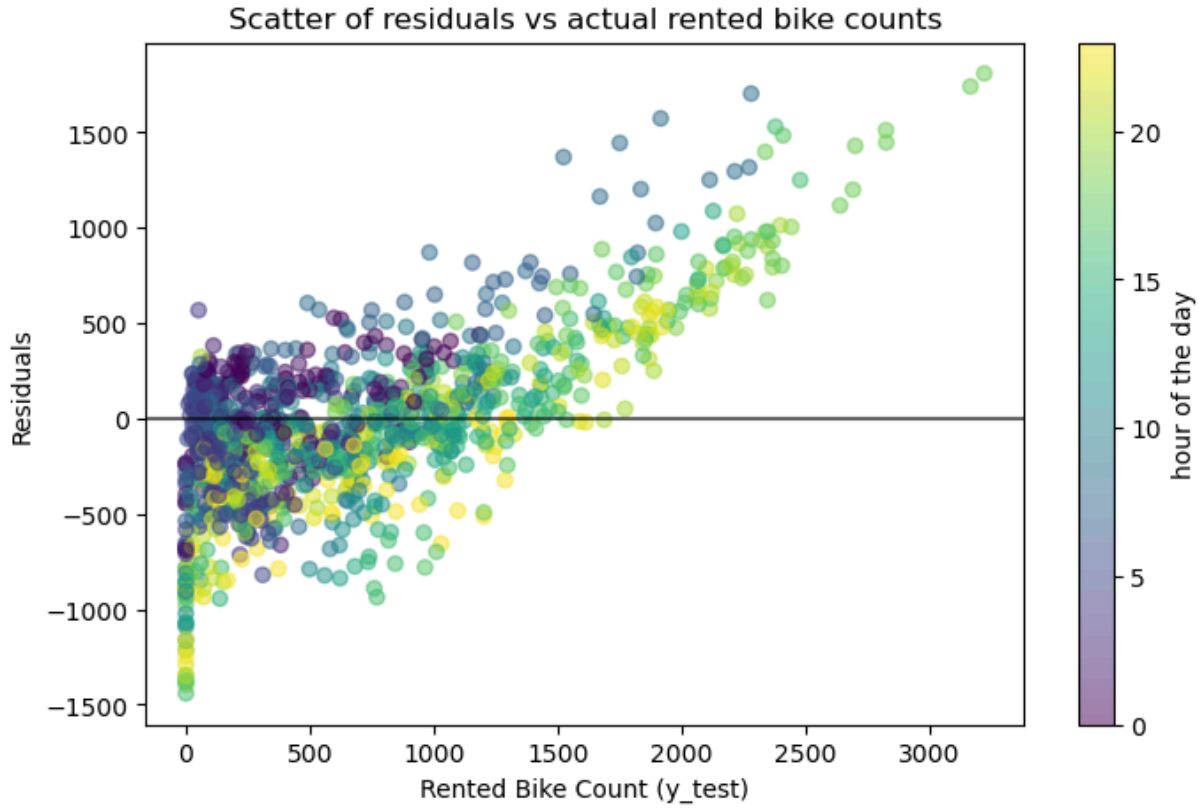
note: whilst these results are very similar, there are discrepancies due to both the random sampling of rows in our training data set, and the fact that the data set used above contained half the data used in the Seoul Bike Paper, thus it is likely that their predictions were more accurate.

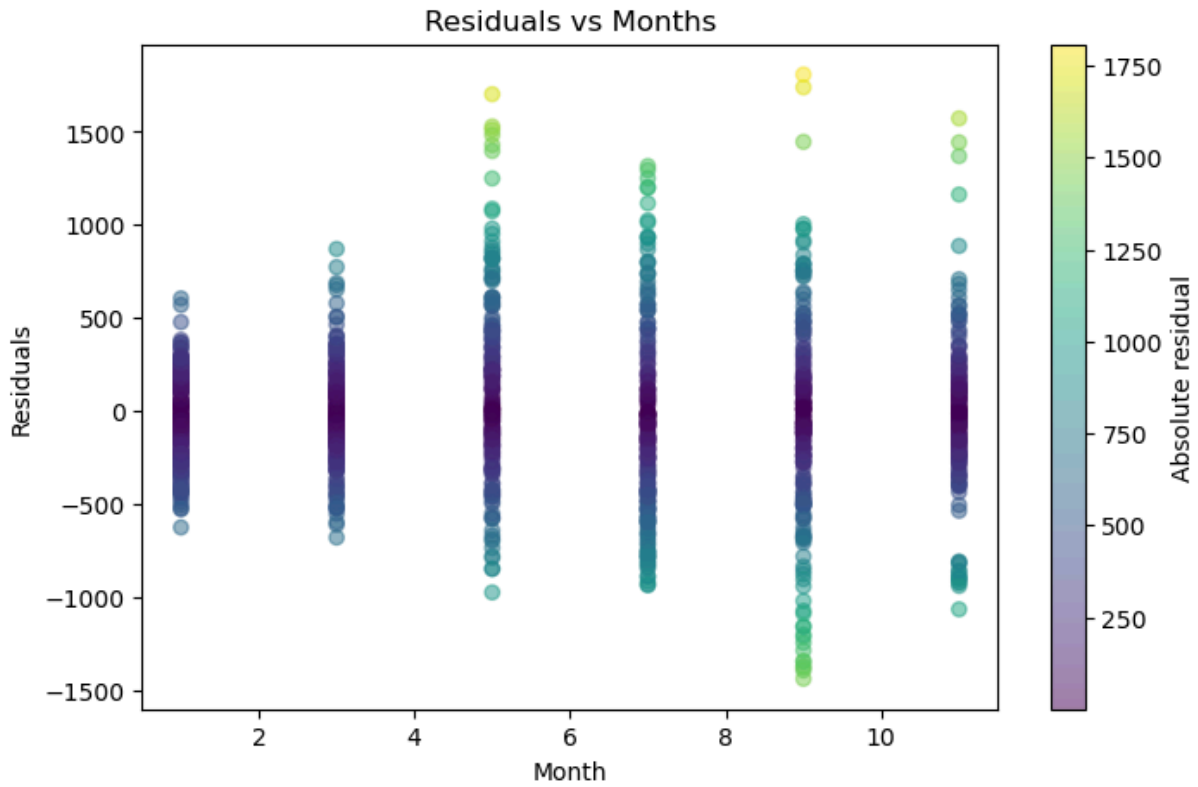
In [4]:

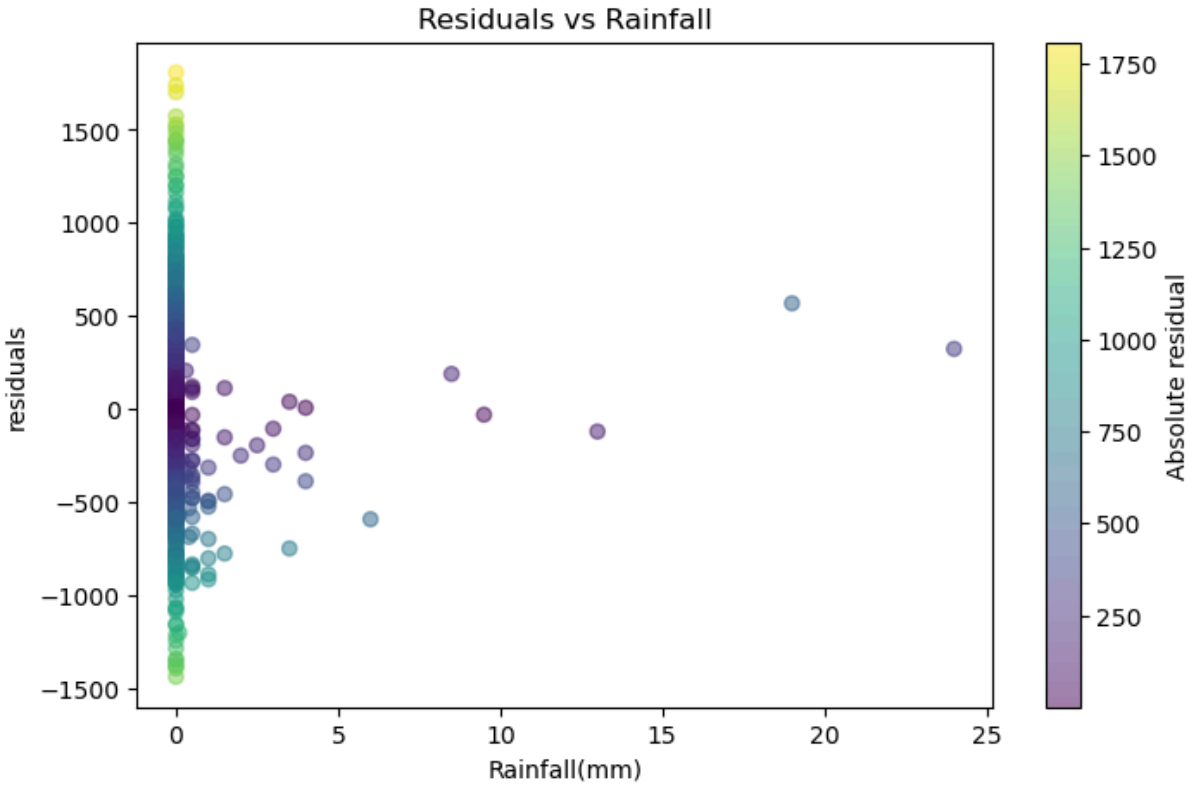
```
#10
# Histograms of y_test and y_pred
plt.figure(figsize=(8,5))
plt.hist(y_test, bins = 50, alpha = 0.5, label = "y_test", color= "blue", edgecolor
plt.hist(y_pred, bins = 50, alpha = 0.5, label = "y_pred", color= "orange", edgecol
plt.xlabel("Rented Bike Count")
plt.ylabel("Frequency")
plt.title("Histogram of actual vs predicted bike counts")
plt.legend()
plt.show()

# Scatter plot of resid as a function of y_test (corresponding to fig 9 of article)
plt.figure(figsize=(8,5))
scatter1 = plt.scatter(y_test, resid, c=X_test["Hour"], cmap= "viridis", alpha = 0.
```

```
plt.axhline(0, color = "black", alpha = 0.6)
plt.xlabel("Rented Bike Count (y_test)")
plt.ylabel("Residuals")
plt.title("Scatter of residuals vs actual rented bike counts")
plt.colorbar(label = "hour of the day")
plt.show()

# Scatter plot of resid vs month
plt.figure(figsize= (8,5))
scatter2 = plt.scatter(X_test["Month"], resid, c=np.abs(resid), cmap = "viridis", a
plt.colorbar(label = "Absolute residual")
plt.xlabel("Month")
plt.ylabel("Residuals")
plt.title("Residuals vs Months")
plt.show()

# Scatter plot of resid vs rainfall
plt.figure(figsize = (8,5))
scatter3 = plt.scatter(X_test["Rainfall(mm)"], resid, c = np.abs(resid), cmap = "vi
plt.colorbar(label = "Absolute residual")
plt.xlabel("Rainfall(mm)")
plt.ylabel("residuals")
plt.title("Residuals vs Rainfall")
plt.show()
```




Scatter of residuals vs actual rented bike counts
Residuals
Rented Bike Count (y_test)
hour of the day
1500
1000
500
0
−500
−1000
−1500
0
500
1000
1500
2000
2500
3000
20
15
10
5
0


Residuals vs Months
Residuals
Month
Absolute residual
1500
1000
500
0
−500
−1000
−1500
2
4
6
8
10
1750
1500
1250
1000
750
500
250



- The histogram shows us that the model is most accurate in the region of 500-1750 rented bike count however the model does an incredibly poor job at predicting rented bike count when it is between 0-500, as evident from the great difference between "y_test" and "y_pred" far below it (this perhaps indicates underfitting of the model). Furthermore the model predicts negative values for rented bike count, which is problematic since this of course does not match the real world data. We may consider that a straight line is too simple for the datas complexity.
- The first scatter plot tells a similar story as the histogram. We can see that the residuals are not well dispersed across the horizontal axis, which again indicates that the models accuracy is inconsistent depending on the volume of bikes being rented. Acknowledge that a superior model would depict random noise centred at zero, with roughly the same spread from left to right. One observation that we get from this scatter plot which is not evident in the histogram is that the colormap reveals that the model is more accurate at predicting during early hours i.e. closer to early morning results in smaller residuals - suggesting its "easier" for the model to predict a low number when the answer is zero than to predict a high number when the answer is massive.
- Immediately we notice in the second scatter plot that the model is more accurate at predicting rented bike count in the months of January and March (there is no data for even months) and much less accurate on the months of april and september. Reasons for the latter may be due to this being the time of school holidays, thus schedules and daily activities are more chaotic and less routine for many households/
- The final scatter plot depicts a thick line at 0mm of Rainfall - it is worth explaining that this does not by itself suggest that the model is wildly inaccurate at predicting rented

bike counts when there is no rain, rather it is simply a reflection of the fact that there are proportionately a lot more data points at 0mm representing days without rain. If we ignore this line, what is left behind suggests that the model is better at predicting the rented bike count when there is rain (comparing with scatter 1).

Section A2:

1.

$$3J = ((m+c)-1)^2 + ((2m+c)-4)^2 + ((3m+c)-5)^2$$

$$\begin{aligned} 3J = &(m^2 + 2mc - 2m + c^2 - 2c + 1) \\ &+ (4m^2 + 4mc - 16m + c^2 - 8c + 16) \\ &+ (9m^2 + 6mc - 30m + c^2 - 10c + 25) \end{aligned}$$

$$J = \frac{14m^2 + 12cm - 48m + 3c^2 - 20c + 42}{3}$$

2.

$$\frac{\partial J}{\partial m} = \frac{28m + 12c - 48}{3}$$

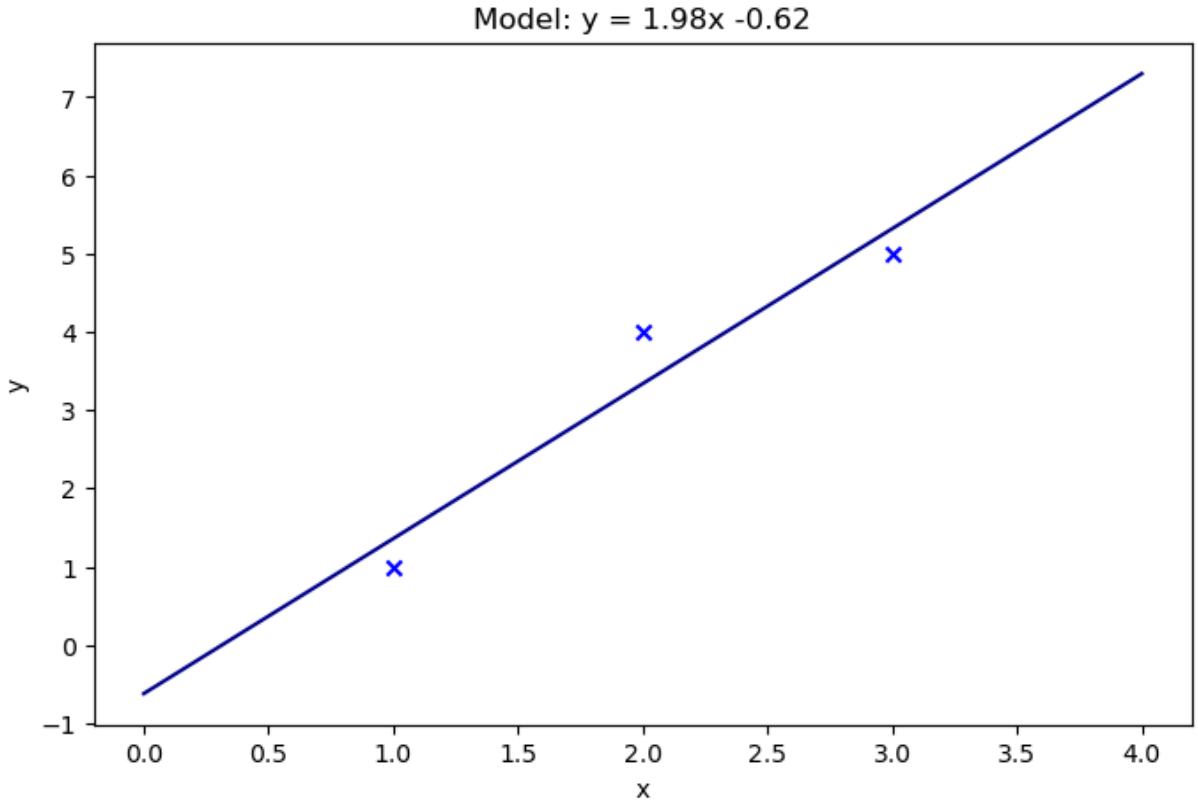
$$\frac{\partial J}{\partial c} = \frac{12m + 6c - 20}{3}$$

In [7]:

```
#3
def J(m,c):
    return (14*m**2 + 12*c*m - 48*m + 3*c**2 -20*c +42)/3

def partials(m,c):
    # this of course calculates the gradient
    pm = (28*m + 12*c - 48)/3
    pc = (12*m + 6*c - 20)/3
    return pm, pc


# setting the seed for reprocible results
np.random.seed(100)

def model1(alpha, num_iterations):
    #initialising m and c
    m = np.random.uniform(-2,2)
    c = np.random.uniform(-2,2)

    print(f"Initial parameters: m = {m}, c = {c}")

    for i in range(1, num_iterations+1):
        pm, pc = partials(m,c)
```

```
        #updating parameters
        m -= alpha * pm
        c -= alpha * pc

        #periodically printing cost
        if i % 100 == 0:
            print(f"Iteration {i} -> Cost = {J(m,c)}")

    return m,c

mOptimal, cOptimal = model1(alpha = 0.01, num_iterations = 1000)
print(f"the optimised parameters are: m = {mOptimal}, c = {cOptimal}")

#4
#plotting
xpoints = np.array([1,2,3])
ypoints = np.array([1,4,5])

xarray = np.linspace(0,4,100)
yarray = mOptimal*xarray+cOptimal

plt.figure(figsize = (8,5))
plt.scatter(xpoints, ypoints, color = "blue", marker = "x")
plt.plot(xarray, yarray, color = "darkblue")
plt.xlabel("x")
plt.ylabel("y")
plt.title(f"Model: y = {mOptimal:.2f}x {cOptimal:.2f}")
plt.show()
```

```
Initial parameters: m = 0.17361976716386174, c = -0.8865224596248154
Iteration 100 -> Cost = 0.24341556411791032
Iteration 200 -> Cost = 0.23531841991310407
Iteration 300 -> Cost = 0.2303148771383121
Iteration 400 -> Cost = 0.22722299153990613
Iteration 500 -> Cost = 0.22531239399254824
Iteration 600 -> Cost = 0.22413176072789773
Iteration 700 -> Cost = 0.223402201063422
Iteration 800 -> Cost = 0.2229513774927483
Iteration 900 -> Cost = 0.22267279590199487
Iteration 1000 -> Cost = 0.22250064939557035
the optimised parameters are: m = 1.9806200897404957, c = -0.622611575405206
```



5. The learning rate, which can be thought of as the 'step size' determines whether the optimal paramters are overshot (if its too high) or if the model is too slow (when the learning rate is too low, resulting in many iterations to reach the optimal). Hence, the number of iterations for a good model depends inversely on the learning rate. Furthermore, adding too many iterations may become inefficient since there becomes a point where the models parameters plateau or just fluctuate without converging to a more accurate value. In order to test the accuracy of the above claims we could run:
   print(model1(0.5,10))
   print(model1(0.5,100))
   print(model1(0.5,500))
   print(model1(0.5,1000))
   print(model1(0.01,10))
   print(model1(0.01,100))
   print(model1(0.01,500))
   print(model1(0.01,1000))
   (The results do not differ from what is described above, so the code will not be run)